

OpenCode 導入ガイド

1 OpenCode とは

OpenCode は、ターミナルで動く AI コーディングアシスタントです。プロジェクトのコードを理解し、質問に答えたり、コードの生成・修正・説明を対話的に行います。

2 この演習での使い道

- コンパイラ実装中に「なぜこのエラーが出るのか」を相談する
- 「この部分のコードが何をしているか」を説明してもらう
- AST の構造やアセンブリ生成の考え方を壁打ちする

注意: OpenCode は教員の代わりではありません。生成されたコードは必ず自分で理解し、テストで動作確認してください。AI の出力を鵜呑みにせず、「なぜ動くのか」を考えることが学習の本質です。

3 インストール

3.1 1. Node.js が入っているか確認

```
node --version
```

v18.0.0 以上が表示されれば OK です。入っていない場合は以下でインストールします。

```
# Ubuntu の場合
sudo apt update
sudo apt install nodejs npm
```

Ubuntu 22.04 の標準パッケージは古いバージョンの場合があります。その場合は [NodeSource](#) の手順で最新版を入れてください。

3.2 2. OpenCode をインストール

```
sudo npm install -g opencode-ai
```

3.3 3. 動作確認

```
opencode --version
```

バージョン番号が表示されればインストール成功です。

4 基本的な使い方

4.1 プロジェクトで起動

```
cd workbook/  
opencode
```

起動すると対話モードになり、プロンプトが表示されます。

4.2 質問する

日本語で質問を入力し、Enter キーを押すだけです。

> この mycc.py の codegen 関数は何をしていますか？

OpenCode はプロジェクト内のファイルを読み取り、質問に回答します。

4.3 ファイルを読ませる

特定のファイルについて質問したい場合は、ファイル名を指定します。

> sessions/03_arithmetic_codegen/mycc.py のコードを説明してください

4.4 コードを書いてもらう

「～を実装したい」と指示すると、コードを生成します。

> if 文の else 節をコード生成するメソッドを追加してください

生成されたコードは、そのままファイルに適用するかどうかを確認されます。

生成されたコードは必ずテスト (`python3 scaffold/test_runner.py`) で動作確認してください。AI の出力には誤りが含まれることがあります。

5 コンパイラ開発での活用例

5.1 例 1: エラーの原因を調べる

コンパイル結果のアセンブリが期待通りにならないとき、コードと実行結果を OpenCode に見せて相談できます。

> `test_runner.py` で `while` のテストが失敗します。

> 生成されたアセンブリを見て、どこが間違っているか指摘してください。

5.2 例 2: コードの構造を理解する

スキャフォールドのコードは教員が書いたものです。OpenCode に「このクラスは何をしているか」「この関数の引数は何か」を質問すると、素早く理解できます。

> `scaffold/parser.py` の `parse_expr` 関数は何を返しますか？

5.3 例 3: 実装の方針を相談する

「配列のコード生成をどう設計すればいいか」といった設計レベルの相談もできます。

> 配列 `a[3]` の宣言をスタック上に確保するコードを生成したいです。

> どのようなアセンブリを出力すればいいですか？

6 注意点

- OpenCode は**教員の代わりにはなりません**。わからないことは教員に質問してください。
- AI の出力は **参考情報** として扱い、必ず自分で検証してください。
- コードをそのままコピーするのではなく、**なぜそうなるのか**を理解することが重要です。
- 機密情報や API キーなどをプロジェクトに含めないでください。